Utica University | Department of Computer Science | Fall 2026
CSC 618: Information Retrieval | Week 10 Assignment

Self Study 1: Query Experimentation and Evaluation in Apache Lucene

Suman KC | November 2026

1. Summary

I ran eight Lucene query classes over the 1400-document Cranfield collection for all 225 topics and compared them with effectiveness metrics, KLD and JSD. Scoring and query scope mattered more than match breadth: constant-score classes (Prefix, Wildcard, Range, MatchAll) gave weak rankings, and requiring one rare term was costly (MAP 0.116 for Boolean with a required term against 0.289 for an OR of all terms). JSD to the relevant documents tracked MAP closely (Spearman $\rho = -0.976$ across the 10 conditions).

2. Setup and implementation

Engine	Apache Lucene 10.5.0, Java 21, in-memory index; `EnglishAnalyzer` (stop words, Porter stemming); BM25 (k_1=1.2, b=0.75)
Collection	Cranfield: 1400 aeronautics abstracts, 225 topics, binary qrels (5172 stemmed terms, mean 102.3 terms per document)
Evaluation	Top 100 results per topic and type, scored against qrels; Python 3 metric code
Code	Appendix A; complete project in `csc618-week10-ir-evaluation.zip`

Naming note. Lucene no longer has a class called `RangeQuery`. I used its two replacements: `TermRangeQuery` (text) and `IntPoint.newRangeQuery` (numeric). Wildcard, prefix and fuzzy terms bypass the analyzer, so patterns must be written against stems, e.g. `turbulanc`, not `turbulence`.

3. Query generation

Hand-picked queries cannot be compared fairly over 225 topics, so each class was generated by a fixed rule that uses only the topic text and corpus statistics, never the qrels. Let t^* be the topic term with the highest IDF.

Table 1: Generation rule per query class.

Label	Rule (Lucene class)
Term	t^* (`TermQuery`)
Phrase	Adjacent topic bigram with highest IDF sum and at least one match, slop 0 (`PhraseQuery`)
Range-Numeric	Document length between the collection quartiles, 62–134 terms; independent of the topic (`IntPoint.newRangeQuery`)
Range-Lexical	Terms from the first 4 characters of t^* up to t^* (`TermRangeQuery`)
Boolean	MUST t^*, SHOULD every other topic term (`BooleanQuery`)
Prefix	First 4 characters of t^* then $\star$ (`PrefixQuery`)
Wildcard	First 3 characters, $\star$, last character of t^* (`WildcardQuery`)
Fuzzy	t^* at edit distance 2 (`FuzzyQuery`)
MatchAll	Every document (`MatchAllDocsQuery`)
OR-Baseline	SHOULD all topic terms: a reference condition, not a ninth class (`BooleanQuery`)

4. Behaviour of each class on hand-written queries

Table 2: Demonstration queries. "Scores" is the number of distinct scores in the top 100; 1 means constant score, so order is simply document ID.

Query	Hits	Scores	Top score	Top 5 document IDs
TermQuery `boundari`	470	62	0.952	4, 899, 1154, 335, 671
PhraseQuery `"boundari layer"`	367	69	2.015	4, 899, 671, 1149, 1225
PhraseQuery `"layer boundari" (reversed)`	0	0	0.000	–
PhraseQuery slop=2 `"layer boundari"`	367	72	1.609	4, 376, 899, 671, 1149
TermRangeQuery `[bound TO boundz]`	482	1	1.000	1, 2, 3, 4, 7
IntPoint range `len in [62,134]`	710	1	1.000	1, 2, 6, 8, 11
BooleanQuery `+boundari +layer -transit`	307	80	2.015	4, 899, 671, 1149, 1225
PrefixQuery `aero*`	230	1	1.000	1, 5, 11, 12, 13
WildcardQuery `t?rbul*`	145	1	1.000	7, 9, 16, 17, 40
FuzzyQuery `turbulanc~2 (misspelt)`	4	4	2.532	74, 651, 257, 822
TermQuery `turbulanc (misspelt)`	0	0	0.000	–
MatchAllDocsQuery	1400	1	1.000	1, 2, 3, 4, 5

Word order matters: the reversed phrase finds nothing until slop 2 restores all 367 hits. A misspelling returns nothing as a `TermQuery` but 4 documents as a `FuzzyQuery`.

5. Effectiveness over all 225 topics

Table 3: Mean per-topic results. No topic returned zero hits for any class.

Query type	Mean hits	P@3	P@10	Recall@100	AP	RR	nDCG@10
Term	18.5	0.111	0.078	0.190	0.086	0.191	0.130
Phrase	5.9	0.154	0.092	0.170	0.102	0.273	0.157
Range-Numeric	710.0	0.004	0.009	0.075	0.005	0.022	0.008
Range-Lexical	40.0	0.065	0.053	0.195	0.064	0.114	0.086
Boolean	18.5	0.164	0.094	0.190	0.116	0.270	0.169
Prefix	76.3	0.062	0.048	0.198	0.056	0.111	0.073
Wildcard	55.8	0.084	0.056	0.189	0.069	0.131	0.095
Fuzzy	426.2	0.076	0.060	0.241	0.068	0.155	0.097
MatchAll	1400.0	0.006	0.004	0.093	0.005	0.017	0.004
OR-Baseline	968.8	0.357	0.228	0.732	0.289	0.520	0.374

6. KLD and JSD

All divergences use $\log_2$ (bits), so JSD lies in $[0,1]$:

$$D_{KL}(P\|Q) = \sum_x P(x)\log_2\frac{P(x)}{Q(x)}, \qquad JSD(P,Q) = \tfrac{1}{2}D_{KL}(P\|M) + \tfrac{1}{2}D_{KL}(Q\|M), \quad M = \tfrac{1}{2}(P+Q).$$

I used two kinds of distribution, averaged across topics:

- **Term level.** The language model of a document set is the mean of its documents' term distributions, mixed with the collection model C at λ=0.1 so no probability is zero. I compare the relevant documents R and the collection C with the top 10 results T.

- **Document level.** A ranked list becomes a distribution over documents, $p(d) \propto 1/\log_2(1+\text{rank}(d))$, smoothed with 1% uniform mass. This reflects both overlap and rank order.

Table 4: Divergences. Lower R columns mean results closer to relevant text; higher C columns mean results more specialised than the collection. The final two columns compare with OR-Baseline.

Query type	KLD($R\|\|T$)	JSD(R,T)	KLD($T\|\|C$)	JSD(T,C)	Doc KLD(base||type)	Top-10 overlap
Term	2.723	0.425	1.718	0.328	8.747	0.214
Phrase	2.901	0.428	2.603	0.457	9.284	0.183
Range-Numeric	3.342	0.497	1.291	0.270	9.739	0.006
Range-Lexical	2.791	0.435	1.458	0.287	8.783	0.147
Boolean	2.534	0.400	1.638	0.315	8.694	0.286
Prefix	2.851	0.440	1.368	0.273	8.801	0.118
Wildcard	2.762	0.431	1.495	0.292	8.820	0.150
Fuzzy	2.775	0.434	1.435	0.287	8.582	0.148
MatchAll	3.518	0.510	1.379	0.293	9.680	0.006
OR-Baseline	1.680	0.289	1.488	0.301	0.000	1.000

Table 5: Mean pairwise document-level JSD between ranked lists (0 = identical, 1 = disjoint).

	Term	Phr	RngN	RngL	Bool	Pref	Wild	Fuz	All	OR
Term	0.00	0.79	0.96	0.20	0.04	0.31	0.18	0.54	0.96	0.76
Phrase	0.79	0.00	0.98	0.84	0.76	0.86	0.83	0.88	0.98	0.81
Range-Numeric	0.96	0.98	0.00	0.93	0.96	0.91	0.93	0.91	0.39	0.92
Range-Lexical	0.20	0.84	0.93	0.00	0.20	0.14	0.22	0.58	0.93	0.78
Boolean	0.04	0.76	0.96	0.20	0.00	0.31	0.18	0.55	0.96	0.74
Prefix	0.31	0.86	0.91	0.14	0.31	0.00	0.21	0.59	0.90	0.79
Wildcard	0.18	0.83	0.93	0.22	0.18	0.21	0.00	0.57	0.92	0.78
Fuzzy	0.54	0.88	0.91	0.58	0.55	0.59	0.57	0.00	0.91	0.80
MatchAll	0.96	0.98	0.39	0.93	0.96	0.90	0.92	0.91	0.00	0.91
OR-Baseline	0.76	0.81	0.92	0.78	0.74	0.79	0.78	0.80	0.91	0.00

Sensitivity. The term-level KLD order held at λ=0.01 (ρ=0.988) and λ=0.5 (ρ=0.964), and the document-level order at 0.1% and 10% smoothing (ρ=1.000 for both). Absolute values do move: document KLD for Term is 11.70 bits at 0.1% and 5.30 at 10%. KLD values should therefore only be compared under one smoothing setting.

7. Key observations

1. **Scoring decides rank quality.** Prefix returns slightly more relevant documents than Term (Recall@100 0.198 vs 0.190) but has lower MAP (0.056 vs 0.086), because its constant score leaves results in document-ID order.
2. **Adding optional clauses improves order without changing the matches.** Boolean and Term match the same documents (mean 18.5 hits), and their JSD is only 0.043, but MAP rises from 0.086 to 0.116.
3. **Phrase is the most precise narrow query.** It has the fewest hits (5.9) and the highest RR of the eight classes (0.273). Its results are also the most specialised (KLD($T\|\|C$) = 2.603), yet this is not the same as being on-topic: KLD($R\|\|T$) is 2.901, worse than Term.
4. **Fuzzy trades precision for recall.** It has the best Recall@100 of the eight classes (0.241) with 426.2 mean hits. Edit distance 2 is a broad match on short stems, so it is best kept for misspellings.
5. **MatchAll and Range-Numeric set the floor.** Both ignore the topic and have MAP near 0.005; they are

useful as filters or sanity checks, not as ranking queries. Their mutual JSD (0.393) is the only low off-diagonal value in their rows, since both return results in document-ID order.

6. **KLD is directional.** Against OR-Baseline at term level, Phrase scores 2.363 bits in one direction and 1.965 in the other. JSD is symmetric and bounded, so it is easier to read as a distance; KLD is useful when one distribution is clearly the reference.
7. **Divergence and effectiveness agree here, with a caveat.** JSD(R,T) and MAP have $\rho = -0.976$, but that is across only 10 conditions, including the two floor conditions. Divergence also needs relevant documents to compute, so it complements rather than replaces qrel-based metrics.

8. Limitations

The rule-generated queries are simple, and rare t^* terms can degrade the classes built on them: for topic 1, t^* is `obei`, making Prefix, Wildcard and Range-Lexical nearly equal to Term. `EnglishAnalyzer` keeps query words such as `what`, which inflates OR-Baseline hit counts. Qrels are binary, and unjudged documents count as non-relevant. Pairwise JSD on top-100 lists also reflects the arbitrary document-ID order of constant-score classes.

References

[1] Apache Lucene, https://lucene.apache.org/. [2] S. Kullback and R. Leibler, "On information and sufficiency," *Ann. Math. Statist.*, 22(1), 1951. [3] J. Lin, "Divergence measures based on the Shannon entropy," *IEEE Trans. Inf. Theory*, 37(1), 1991. [4] Cranfield collection (TREC-format copy), https://github.com/oussbenk/cranfield-trec-dataset.

Appendix A: Code

LuceneExperiment.java (indexing, query generation, retrieval runs)

```
import java.io.IOException;
import java.io.PrintWriter;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.util.ArrayList;
import java.util.Collections;
import java.util.HashSet;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Locale;
import java.util.Map;
import java.util.Set;
import org.apache.lucene.analysis.Analyzer;
import org.apache.lucene.analysis.TokenStream;
import org.apache.lucene.analysis.en.EnglishAnalyzer;
import org.apache.lucene.analysis.tokenattributes.CharTermAttribute;
import org.apache.lucene.analysis.tokenattributes.PositionIncrementAttribute;
import org.apache.lucene.document.Document;
import org.apache.lucene.document.Field;
import org.apache.lucene.document.IntPoint;
import org.apache.lucene.document.StringField;
import org.apache.lucene.document.TextField;
import org.apache.lucene.index.DirectoryReader;
import org.apache.lucene.index.IndexReader;
import org.apache.lucene.index.IndexWriter;
import org.apache.lucene.index.IndexWriterConfig;
import org.apache.lucene.index.MultiTerms;
import org.apache.lucene.index.StoredFields;
import org.apache.lucene.index.Term;
import org.apache.lucene.search.BooleanClause;
import org.apache.lucene.search.BooleanQuery;
import org.apache.lucene.search.FuzzyQuery;
import org.apache.lucene.search.IndexSearcher;
import org.apache.lucene.search.MatchAllDocsQuery;
import org.apache.lucene.search.MatchNoDocsQuery;
import org.apache.lucene.search.PhraseQuery;
import org.apache.lucene.search.PrefixQuery;
import org.apache.lucene.search.Query;
import org.apache.lucene.search.ScoreDoc;
import org.apache.lucene.search.TermQuery;
import org.apache.lucene.search.TermRangeQuery;
import org.apache.lucene.search.WildcardQuery;
import org.apache.lucene.search.similarities.BM25Similarity;
import org.apache.lucene.search.similarities.ClassicSimilarity;
import org.apache.lucene.search.similarities.LMDirichletSimilarity;
import org.apache.lucene.search.similarities.LMJelinekMercerSimilarity;
import org.apache.lucene.search.similarities.Similarity;
import org.apache.lucene.store.ByteBuffersDirectory;
import org.apache.lucene.store.Directory;

/**
 * Indexes Cranfield in Lucene, then produces all ranked result files.
 *
 * <p>Self Study 1: eight query classes per topic (top 100). Part C: retrieval models (top 1000).
 *
 * <p>Usage: java LuceneExperiment &lt;dataDir&gt; &lt;resultsDir&gt;
 */
public class LuceneExperiment {
  private static final String FIELD = "contents";
  private static final String LENGTH_FIELD = "len";
  private static final Analyzer ANALYZER = new EnglishAnalyzer();

  /** One analysed token and its position (positions keep gaps left by removed stop words). */
  record Token(String term, int position) {}
```

```
/** Document-length summary used by the numeric range query. */
record LengthStats(double mean, int q1, int q3) {}

public static void main(String[] args) throws Exception {
  Path dataDir = Paths.get(args[0]);
  Path resultsDir = Paths.get(args[1]);
  Files.createDirectories(resultsDir);

  Directory directory = new ByteBuffersDirectory();
  LengthStats lengths = buildIndex(dataDir, resultsDir, directory);
  List<String[]> queries = readQueries(dataDir);

  try (IndexReader reader = DirectoryReader.open(directory)) {
    writeIndexStats(reader, lengths, resultsDir);
    IndexSearcher bm25Searcher = new IndexSearcher(reader);
    bm25Searcher.setSimilarity(new BM25Similarity());

    runQueryTypeExperiment(reader, bm25Searcher, queries, lengths, resultsDir);
    runDemoQueries(reader, bm25Searcher, lengths, resultsDir);
    runRetrievalModels(reader, queries, resultsDir);
  }
}

// ---------------------------------------------------------------- Indexing

/** Indexes every document and also saves its analysed terms for the KLD/JSD calculations. */
private static LengthStats buildIndex(Path dataDir, Path resultsDir, Directory directory)
    throws IOException {
  List<Integer> lengths = new ArrayList<>();
  IndexWriterConfig config = new IndexWriterConfig(ANALYZER).setSimilarity(new BM25Similarity());

  try (IndexWriter writer = new IndexWriter(directory, config);
      PrintWriter analysed = new PrintWriter(resultsDir.resolve("docs_analyzed.tsv").toFile())) {
    for (String line : Files.readAllLines(dataDir.resolve("docs.tsv"))) {
      String[] parts = line.split("\t", -1);
      String id = parts[0];
      String title = parts[1];
      String text = parts[2];
      // The <text> field already begins with the title; two documents have no text at all.
      String body = text.isEmpty() ? title : text;

      List<Token> tokens = analyze(body);
      lengths.add(tokens.size());

      Document document = new Document();
      document.add(new StringField("id", id, Field.Store.YES));
      document.add(new TextField(FIELD, body, Field.Store.NO));
      document.add(new IntPoint(LENGTH_FIELD, tokens.size()));
      writer.addDocument(document);

      StringBuilder row = new StringBuilder(id).append('\t');
      for (Token token : tokens) {
        row.append(token.term()).append(' ');
      }
      analysed.println(row.toString().trim());
    }
  }

  Collections.sort(lengths);
  int q1 = lengths.get(lengths.size() / 4);
  int q3 = lengths.get(3 * lengths.size() / 4);
  double mean = lengths.stream().mapToInt(Integer::intValue).average().orElse(0);
  return new LengthStats(mean, q1, q3);
}

private static void writeIndexStats(IndexReader reader, LengthStats lengths, Path resultsDir)
    throws IOException {
  try (PrintWriter out = new PrintWriter(resultsDir.resolve("index_stats.txt").toFile())) {
    out.printf(
```

```
        Locale.ROOT,
        "lucene=%s docs=%d vocab=%d tokens=%d meanLen=%.2f q1=%d q3=%d%n",
        org.apache.lucene.util.Version.LATEST,
        reader.numDocs(),
        MultiTerms.getTerms(reader, FIELD).size(),
        reader.getSumTotalTermFreq(FIELD),
        lengths.mean(),
        lengths.q1(),
        lengths.q3());
  }
}

// ------------------------------------------------------------------ Helpers

/** Applies the same analysis to queries as to documents (stop words, Porter stemming). */
private static List<Token> analyze(String text) throws IOException {
  List<Token> tokens = new ArrayList<>();
  try (TokenStream stream = ANALYZER.tokenStream(FIELD, text)) {
    CharTermAttribute term = stream.addAttribute(CharTermAttribute.class);
    PositionIncrementAttribute increment = stream.addAttribute(PositionIncrementAttribute.class);
    stream.reset();
    int position = -1;
    while (stream.incrementToken()) {
      position += increment.getPositionIncrement();
      tokens.add(new Token(term.toString(), position));
    }
    stream.end();
  }
  return tokens;
}

private static List<String[]> readQueries(Path dataDir) throws IOException {
  List<String[]> queries = new ArrayList<>();
  for (String line : Files.readAllLines(dataDir.resolve("queries.tsv"))) {
    queries.add(line.split("\t", -1)); // sequential id, original number, text
  }
  return queries;
}

private static Term term(String text) {
  return new Term(FIELD, text);
}

/** OR of all query terms: a document needs at least one of them to match. */
private static Query orQuery(List<Token> tokens) {
  BooleanQuery.Builder builder = new BooleanQuery.Builder();
  for (Token token : tokens) {
    builder.add(new TermQuery(term(token.term())), BooleanClause.Occur.SHOULD);
  }
  return builder.build();
}

/** Inverse document frequency, or -1 when the term is not in the index. */
private static double idf(IndexReader reader, String text) throws IOException {
  int documentFrequency = reader.docFreq(term(text));
  return documentFrequency == 0 ? -1 : Math.log((reader.numDocs() + 1.0) / documentFrequency);
}

/** Runs a query and writes results in TREC format: qid Q0 docid rank score tag. */
private static void writeRun(
    IndexSearcher searcher, Query query, int queryId, String tag, int depth, PrintWriter out)
    throws IOException {
  StoredFields fields = searcher.getIndexReader().storedFields();
  ScoreDoc[] hits = searcher.search(query, depth).scoreDocs;
  for (int i = 0; i < hits.length; i++) {
    String docId = fields.document(hits[i].doc).get("id");
    out.printf(Locale.ROOT, "%d Q0 %s %d %.6f %s%n", queryId, docId, i + 1, hits[i].score, tag);
  }
}
```

```java
// ------------------------------------------------------------------ Self Study 1

private static String rarestTerm(IndexReader reader, List<Token> tokens) throws IOException {
  String best = null;
  double bestIdf = -1;
  for (Token token : tokens) {
    double value = idf(reader, token.term());
    if (value > bestIdf) {
      bestIdf = value;
      best = token.term();
    }
  }
  return best;
}

/** Adjacent bigram with the highest IDF sum that matches at least one document. */
private static Query bestPhrase(IndexReader reader, IndexSearcher searcher, List<Token> tokens)
    throws IOException {
  PhraseQuery best = null;
  double bestValue = -1e9;
  for (int i = 0; i + 1 < tokens.size(); i++) {
    Token first = tokens.get(i);
    Token second = tokens.get(i + 1);
    if (first.term().equals(second.term())) {
      continue;
    }
    PhraseQuery candidate =
        new PhraseQuery.Builder()
            .add(term(first.term()), 0)
            .add(term(second.term()), second.position() - first.position())
            .build();
    double value = idf(reader, first.term()) + idf(reader, second.term());
    if (searcher.count(candidate) > 0 && value > bestValue) {
      bestValue = value;
      best = candidate;
    }
  }
  return best == null ? new MatchNoDocsQuery("no matching bigram") : best;
}

/** Builds each query class for one topic. Uses topic text and corpus statistics, never qrels. */
private static Map<String, Query> buildQueries(
    IndexReader reader, IndexSearcher searcher, List<Token> tokens, LengthStats lengths)
    throws IOException {
  String rarest = rarestTerm(reader, tokens);
  String prefix = rarest.substring(0, Math.min(4, rarest.length()));
  String wildcard =
      rarest.length() >= 5
          ? rarest.substring(0, 3) + "*" + rarest.charAt(rarest.length() - 1)
          : rarest + "*";

  BooleanQuery.Builder bool = new BooleanQuery.Builder();
  bool.add(new TermQuery(term(rarest)), BooleanClause.Occur.MUST);
  for (Token token : tokens) {
    if (!token.term().equals(rarest)) {
      bool.add(new TermQuery(term(token.term())), BooleanClause.Occur.SHOULD);
    }
  }

  Map<String, Query> queries = new LinkedHashMap<>();
  queries.put("Term", new TermQuery(term(rarest)));
  queries.put("Phrase", bestPhrase(reader, searcher, tokens));
  queries.put("Range-Numeric", IntPoint.newRangeQuery(LENGTH_FIELD, lengths.q1(), lengths.q3()));
  queries.put("Range-Lexical", TermRangeQuery.newStringRange(FIELD, prefix, rarest, true, true));
  queries.put("Boolean", bool.build());
  queries.put("Prefix", new PrefixQuery(term(prefix)));
  queries.put("Wildcard", new WildcardQuery(term(wildcard)));
  queries.put("Fuzzy", new FuzzyQuery(term(rarest), 2));
```

```
  queries.put("MatchAll", new MatchAllDocsQuery());
  queries.put("OR-Baseline", orQuery(tokens)); // reference condition, not a ninth class
  return queries;
}

private static void runQueryTypeExperiment(
    IndexReader reader,
    IndexSearcher searcher,
    List<String[]> topics,
    LengthStats lengths,
    Path resultsDir)
    throws IOException {
  String[] types = {
    "Term", "Phrase", "Range-Numeric", "Range-Lexical", "Boolean",
    "Prefix", "Wildcard", "Fuzzy", "MatchAll", "OR-Baseline"
  };
  Map<String, PrintWriter> runFiles = new LinkedHashMap<>();
  for (String type : types) {
    runFiles.put(type, new PrintWriter(resultsDir.resolve("ss1_" + type + ".run").toFile()));
  }
  try (PrintWriter generated =
          new PrintWriter(resultsDir.resolve("ss1_generated_queries.tsv").toFile());
       PrintWriter counts = new PrintWriter(resultsDir.resolve("ss1_hit_counts.tsv").toFile())) {
    generated.println("qid\ttype\tquery");
    counts.println("qid\ttype\thits");
    for (String[] topic : topics) {
      int id = Integer.parseInt(topic[0]);
      List<Token> tokens = analyze(topic[2]);
      for (var entry : buildQueries(reader, searcher, tokens, lengths).entrySet()) {
        String type = entry.getKey();
        Query query = entry.getValue();
        writeRun(searcher, query, id, type, 100, runFiles.get(type));
        generated.println(id + "\t" + type + "\t" + query);
        counts.println(id + "\t" + type + "\t" + searcher.count(query));
      }
    }
  }
  runFiles.values().forEach(PrintWriter::close);
}

/** Hand-written queries showing how each class behaves, including word order and misspelling. */
private static void runDemoQueries(
    IndexReader reader, IndexSearcher searcher, LengthStats lengths, Path resultsDir)
    throws IOException {
  Map<String, Query> demo = new LinkedHashMap<>();
  demo.put("TermQuery  boundari", new TermQuery(term("boundari")));
  demo.put("PhraseQuery  \"boundari layer\"", new PhraseQuery(FIELD, "boundari", "layer"));
  demo.put(
      "PhraseQuery  \"layer boundari\" (reversed)", new PhraseQuery(FIELD, "layer", "boundari"));
  demo.put(
      "PhraseQuery slop=2  \"layer boundari\"", new PhraseQuery(2, FIELD, "layer", "boundari"));
  demo.put(
      "TermRangeQuery  [bound TO boundz]",
      TermRangeQuery.newStringRange(FIELD, "bound", "boundz", true, true));
  demo.put(
      "IntPoint range  len in [" + lengths.q1() + "," + lengths.q3() + "]",
      IntPoint.newRangeQuery(LENGTH_FIELD, lengths.q1(), lengths.q3()));
  demo.put(
      "BooleanQuery  +boundari +layer -transit",
      new BooleanQuery.Builder()
          .add(new TermQuery(term("boundari")), BooleanClause.Occur.MUST)
          .add(new TermQuery(term("layer")), BooleanClause.Occur.MUST)
          .add(new TermQuery(term("transit")), BooleanClause.Occur.MUST_NOT)
          .build());
  demo.put("PrefixQuery  aero*", new PrefixQuery(term("aero")));
  demo.put("WildcardQuery  t?rbul*", new WildcardQuery(term("t?rbul*")));
  demo.put("FuzzyQuery  turbulanc~2 (misspelt)", new FuzzyQuery(term("turbulanc"), 2));
  demo.put("TermQuery  turbulanc (misspelt)", new TermQuery(term("turbulanc")));
  demo.put("MatchAllDocsQuery", new MatchAllDocsQuery());
```

```
    StoredFields fields = reader.storedFields();
    try (PrintWriter out = new PrintWriter(resultsDir.resolve("ss1_demo.tsv").toFile())) {
      out.println("query\thits\tdistinctTop100Scores\ttopScore\ttop5");
      for (var entry : demo.entrySet()) {
        ScoreDoc[] hits = searcher.search(entry.getValue(), 100).scoreDocs;
        Set<Float> distinctScores = new HashSet<>();
        for (ScoreDoc hit : hits) {
          distinctScores.add(hit.score);
        }
        StringBuilder top = new StringBuilder();
        for (int i = 0; i < Math.min(5, hits.length); i++) {
          top.append(i > 0 ? "," : "").append(fields.document(hits[i].doc).get("id"));
        }
        out.printf(
            Locale.ROOT,
            "%s\t%d\t%d\t%.3f\t%s%n",
            entry.getKey(),
            searcher.count(entry.getValue()),
            distinctScores.size(),
            hits.length > 0 ? hits[0].score : 0,
            top);
      }
    }
  }

  // ---------------------------------------------------------------- Part C

  /** Same index, analyzer and queries for every model; only the scoring function changes. */
  private static void runRetrievalModels(
      IndexReader reader, List<String[]> topics, Path resultsDir) throws IOException {
    Map<String, Similarity> models = new LinkedHashMap<>();
    models.put("TF-IDF", new ClassicSimilarity());
    models.put("BM25", new BM25Similarity(1.2f, 0.75f));
    models.put("LM-Dirichlet", new LMDirichletSimilarity(2000f));
    // Sensitivity variants, reported separately from the default-parameter comparison.
    for (float mu : new float[] {50f, 100f, 200f, 500f, 1000f}) {
      models.put("LMD-mu" + (int) mu, new LMDirichletSimilarity(mu));
    }
    for (float lambda : new float[] {0.1f, 0.3f, 0.7f}) {
      models.put("LMJM-l" + lambda, new LMJelinekMercerSimilarity(lambda));
    }
    for (float b : new float[] {0.25f, 0.5f, 1.0f}) {
      models.put("BM25-b" + b, new BM25Similarity(1.2f, b));
    }

    for (var model : models.entrySet()) {
      IndexSearcher searcher = new IndexSearcher(reader);
      searcher.setSimilarity(model.getValue());
      Path file = resultsDir.resolve("partc_" + model.getKey() + ".run");
      try (PrintWriter out = new PrintWriter(file.toFile())) {
        for (String[] topic : topics) {
          Query query = orQuery(analyze(topic[2]));
          writeRun(searcher, query, Integer.parseInt(topic[0]), model.getKey(), 1000, out);
        }
      }
    }
  }
}
```

ss1_eval.py (metrics, KLD, JSD)

```
"""Self Study 1: effectiveness metrics plus KLD and JSD for each Lucene query class.

All divergences use log base 2, so they are in bits and JSD lies in [0, 1].

Usage: python ss1_eval.py <data_dir> <results_dir>
"""
import collections
```

```
import json
import sys

import numpy as np
from scipy.stats import spearmanr

from ir_metrics import (
    average_precision,
    f1,
    mean,
    ndcg_at_k,
    precision_at_k,
    recall,
    reciprocal_rank,
)
from part_c_eval import load_qrels, load_run

TYPES = [
    "Term", "Phrase", "Range-Numeric", "Range-Lexical", "Boolean",
    "Prefix", "Wildcard", "Fuzzy", "MatchAll", "OR-Baseline",
]
BASELINE = "OR-Baseline"
LAMBDA = 0.1  # weight of the collection model in term-level smoothing
EPSILON = 0.01  # weight of the uniform distribution in document-level smoothing


class Collection:
    """Term distributions for every document and for the whole collection."""

    def __init__(self, path):
        docs = {}
        for line in open(path):
            parts = line.rstrip("\n").split("\t")
            docs[parts[0]] = parts[1].split() if len(parts) > 1 else []
        vocabulary = {w: i for i, w in enumerate(sorted({w for t in docs.values() for w in t}))}
        self.ids = list(docs)
        self.position = {doc: i for i, doc in enumerate(self.ids)}
        counts = np.zeros((len(self.ids), len(vocabulary)))
        for doc, terms in docs.items():
            for word in terms:
                counts[self.position[doc], vocabulary[word]] += 1
        self.model = counts.sum(0) / counts.sum()
        self.doc_models = counts / np.maximum(counts.sum(1, keepdims=True), 1)
        self.uniform = np.ones(len(self.ids)) / len(self.ids)

    def language_model(self, doc_ids, lam=LAMBDA):
        """Mean document model mixed with the collection, so every term has P > 0."""
        rows = [self.position[d] for d in doc_ids]
        p = self.doc_models[rows].mean(0) if rows else self.model
        return (1 - lam) * p + lam * self.model

    def rank_distribution(self, ranked, eps=EPSILON):
        """Rank-discounted distribution over documents: p(d) proportional to 1/log2(1+rank)."""
        p = np.zeros(len(self.ids))
        for rank, doc in enumerate(ranked, 1):
            p[self.position[doc]] = 1 / np.log2(rank + 1)
        if p.sum() == 0:
            return self.uniform
        return (1 - eps) * p / p.sum() + eps * self.uniform


def kld(p, q):
    return float(np.sum(p * np.log2(p / q)))


def jsd(p, q):
    m = (p + q) / 2
    return 0.5 * kld(p, m) + 0.5 * kld(q, m)
```

```
def load_hit_counts(path):
    hits = collections.defaultdict(dict)
    for line in list(open(path))[1:]:
        query, query_type, count = line.split("\t")
        hits[query_type][query] = int(count)
    return hits


def effectiveness(ranked, rel, gains):
    return {
        "P10": precision_at_k(ranked, rel, 10),
        "R100": recall(ranked, rel),
        "F1": f1(ranked[:10], rel),
        "P3": precision_at_k(ranked, rel, 3),
        "AP": average_precision(ranked, rel),
        "RR": reciprocal_rank(ranked, rel),
        "nDCG10": ndcg_at_k(ranked, gains, 10),
    }


def summarise_type(query_type, runs, queries, relevant, gains, hits, coll):
    run = runs[query_type]
    per_query = [effectiveness(run.get(q, []), relevant[q], gains[q]) for q in queries]
    row = {key: mean(scores[key] for scores in per_query) for key in per_query[0]}

    kld_rel, jsd_rel, kld_coll, jsd_coll, kld_doc = [], [], [], [], []
    for query in queries:
        ranked = run.get(query, [])
        rel_model = coll.language_model(sorted(relevant[query]))
        top_model = coll.language_model(ranked[:10])
        kld_rel.append(kld(rel_model, top_model))
        jsd_rel.append(jsd(rel_model, top_model))
        kld_coll.append(kld(top_model, coll.model))
        jsd_coll.append(jsd(top_model, coll.model))
        baseline = coll.rank_distribution(runs[BASELINE].get(query, []))
        kld_doc.append(kld(baseline, coll.rank_distribution(ranked)))

    counts = [hits[query_type][q] for q in queries]
    row.update(
        KLD_rel=mean(kld_rel),
        JSD_rel=mean(jsd_rel),
        KLD_coll=mean(kld_coll),
        JSD_coll=mean(jsd_coll),
        KLD_base_doc=mean(kld_doc),
        mean_hits=mean(counts),
        median_hits=float(np.median(counts)),
        zero=sum(count == 0 for count in counts),
    )
    return row


def pairwise_jsd(runs, queries, coll):
    return {
        a: {
            b: mean(
                jsd(
                    coll.rank_distribution(runs[a].get(q, [])),
                    coll.rank_distribution(runs[b].get(q, [])),
                )
                for q in queries
            )
            for b in TYPES
        }
        for a in TYPES
    }


def kld_asymmetry(runs, queries, coll):
```

```
    """KLD in both directions against the baseline, at term level."""
    result = {}
    for query_type in TYPES:
        forward, backward = [], []
        for q in queries:
            base = coll.language_model(runs[BASELINE].get(q, [])[:10])
            other = coll.language_model(runs[query_type].get(q, [])[:10])
            forward.append(kld(base, other))
            backward.append(kld(other, base))
        result[query_type] = (mean(forward), mean(backward))
    return result


def smoothing_sensitivity(table, runs, queries, relevant, coll):
    """Do the KLD orderings hold under other smoothing weights? (Spearman against the default.)"""
    sensitivity = {}
    base_term = [table[t]["KLD_rel"] for t in TYPES]
    for lam in [0.01, 0.5]:
        values = {
            t: mean(
                kld(
                    coll.language_model(sorted(relevant[q]), lam),
                    coll.language_model(runs[t].get(q, [])[:10], lam),
                )
                for q in queries
            )
            for t in TYPES
        }
        rho = spearmanr(base_term, [values[t] for t in TYPES]).statistic
        sensitivity[f"lambda={lam}"] = {"values": values, "rho": float(rho)}

    compared = [t for t in TYPES if t != BASELINE]
    base_doc = [table[t]["KLD_base_doc"] for t in compared]
    for eps in [0.001, 0.1]:
        values = {
            t: mean(
                kld(
                    coll.rank_distribution(runs[BASELINE].get(q, []), eps),
                    coll.rank_distribution(runs[t].get(q, []), eps),
                )
                for q in queries
            )
            for t in compared
        }
        rho = spearmanr(base_doc, [values[t] for t in compared]).statistic
        sensitivity[f"eps={eps}"] = {"values": values, "rho": float(rho)}
    return sensitivity


def main(data, results_dir):
    relevant, gains = load_qrels(f"{data}/qrels.txt")
    queries = sorted(relevant, key=int)
    runs = {t: load_run(f"{results_dir}/ss1_{t}.run") for t in TYPES}
    hits = load_hit_counts(f"{results_dir}/ss1_hit_counts.tsv")
    coll = Collection(f"{results_dir}/docs_analyzed.tsv")

    table = {t: summarise_type(t, runs, queries, relevant, gains, hits, coll) for t in TYPES}
    correlation = spearmanr([table[t]["JSD_rel"] for t in TYPES], [table[t]["AP"] for t in TYPES])
    overlap = {
        t: mean(
            len(set(runs[t].get(q, [])[:10]) & set(runs[BASELINE].get(q, [])[:10])) / 10
            for q in queries
        )
        for t in TYPES
    }
    output = {
        "tab": table,
        "J": pairwise_jsd(runs, queries, coll),
        "asym": kld_asymmetry(runs, queries, coll),
```

```
        "overlap": overlap,
        "corr": {"rho": float(correlation.statistic), "p": float(correlation.pvalue)},
        "V": len(coll.model),
        "lam": LAMBDA,
        "eps": EPSILON,
        "sens": smoothing_sensitivity(table, runs, queries, relevant, coll),
    }
    json.dump(output, open(f"{results_dir}/ss1_results.json", "w"), indent=1)

    columns = ["mean_hits", "P3", "P10", "R100", "AP", "RR", "nDCG10", "KLD_rel", "JSD_rel"]
    print(f"{'type':14}" + "".join(f"{c:>10}" for c in columns))
    for t in TYPES:
        print(f"{t:14}" + "".join(f"{table[t][c]:10.3f}" for c in columns))


if __name__ == "__main__":
    main(sys.argv[1], sys.argv[2])
```

ir_metrics.py

```
"""IR evaluation metrics shared by Part B and Part C.

A ranked list is a list of document IDs, best first. `rel` is the set of relevant IDs.
"""
import math


def precision(ranked, rel):
    return sum(d in rel for d in ranked) / len(ranked) if ranked else 0.0


def recall(ranked, rel):
    return sum(d in rel for d in ranked) / len(rel) if rel else 0.0


def f1(ranked, rel):
    p, r = precision(ranked, rel), recall(ranked, rel)
    return 2 * p * r / (p + r) if p + r else 0.0


def precision_at_k(ranked, rel, k):
    """The denominator stays k even if fewer than k documents were returned."""
    return sum(d in rel for d in ranked[:k]) / k


def reciprocal_rank(ranked, rel):
    for rank, doc in enumerate(ranked, 1):
        if doc in rel:
            return 1 / rank
    return 0.0


def average_precision(ranked, rel):
    """Relevant documents that were not retrieved contribute 0 (division by |rel|)."""
    hits, total = 0, 0.0
    for rank, doc in enumerate(ranked, 1):
        if doc in rel:
            hits += 1
            total += hits / rank
    return total / len(rel) if rel else 0.0


def dcg(gains):
    return sum(gain / math.log2(rank + 1) for rank, gain in enumerate(gains, 1))


def ndcg_at_k(ranked, grades, k):
    """`grades` maps doc -> gain; unjudged documents have gain 0.
```

```
    The ideal list is built from all judged documents, not just the retrieved ones.
    """
    actual = dcg([grades.get(d, 0) for d in ranked[:k]])
    ideal = dcg(sorted(grades.values(), reverse=True)[:k])
    return actual / ideal if ideal else 0.0


def mean(values):
    values = list(values)
    return sum(values) / len(values) if values else 0.0
```